

INTERNAL ONLY!

Do not distribute outside of DataBank

Unity Form Default Values

This document contains instructions and sample code to create a shared Unity form link that populates default field values.

Sample Main Program (and Globals) for Testing:

```
1. private const string BaseUrl = "http://<Host>/AppNet/UnityForm.aspx?d1=<SharedFormHash>";
2. private const string Token = "<IntegrationToken>";
3. private static readonly List<KeyValuePair<string, string>> FormFields = new List<KeyValuePair<string, string>>
4. {
5.     new KeyValuePair<string, string>("<FieldId_1>", "<FieldValue_1>")
6.     // ...
7. };
```

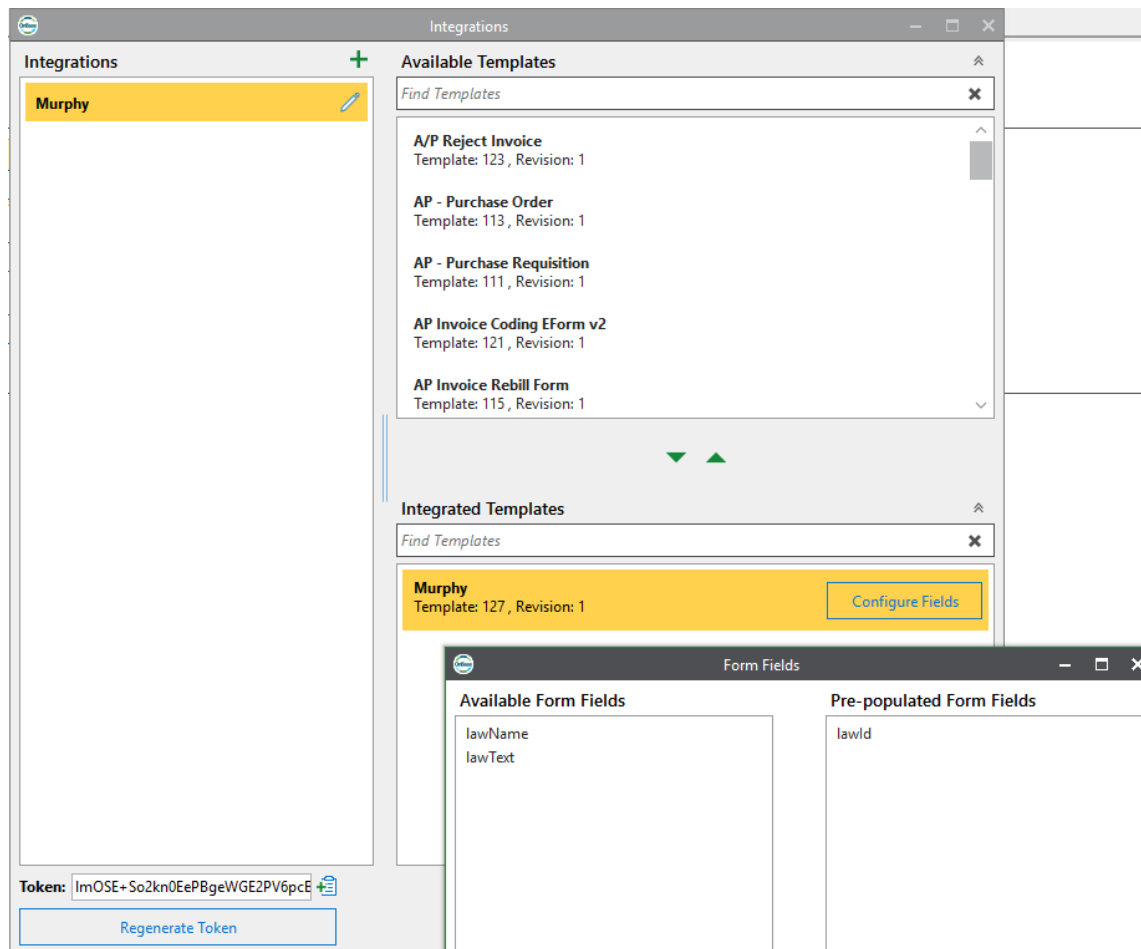
```
1. // Main Method
2. private static void Main()
3. {
4.     try
5.     {
6.         string parameterString = CreateParameterString(FormFields);
7.         byte[] parameterBytes = StringToBytes(parameterString);
8.         string hash = GenerateHash(Token, parameterBytes);
9.         string url = GenerateUrl(BaseUrl, parameterString, hash);
10.        Process.Start(url);
11.    }
12.    catch (Exception ex)
13.    {
14.        while (ex != null)
15.        {
16.            Console.WriteLine(ex);
17.            ex = ex.InnerException;
18.        }
19.    }
20.    finally
21.    {
22.        Console.WriteLine("Done! Press <ENTER> to exit...");
23.        Console.ReadLine();
24.    }
25. }
```

INTERNAL ONLY!

Do not distribute outside of DataBank

1. Configure an integration for the form that has a token available.


Integrations



The screenshot displays two overlapping windows from a software application. The background window is titled 'Integrations' and features a sidebar with a list of integrations, where 'Murphy' is selected and highlighted in yellow. The main area of this window is divided into two sections: 'Available Templates' and 'Integrated Templates'. The 'Available Templates' section contains a search bar and a list of five templates: 'A/P Reject Invoice' (Template: 123, Revision: 1), 'AP - Purchase Order' (Template: 113, Revision: 1), 'AP - Purchase Requisition' (Template: 111, Revision: 1), 'AP Invoice Coding EForm v2' (Template: 121, Revision: 1), and 'AP Invoice Rebill Form' (Template: 115, Revision: 1). The 'Integrated Templates' section also has a search bar and currently displays the 'Murphy' integration (Template: 127, Revision: 1) with a 'Configure Fields' button next to it. At the bottom of the 'Integrations' window, there is a 'Token' field containing the value 'ImOSE+So2kn0EePBgeWGE2PV6pcE' and a 'Regenerate Token' button. The foreground window is titled 'Form Fields' and is divided into two panes. The left pane, 'Available Form Fields', lists 'lawName' and 'lawText'. The right pane, 'Pre-populated Form Fields', lists 'lawId'.

INTERNAL ONLY!

Do not distribute outside of DataBank

2. Generate a shared form URL for a form using the Forms Designer. Be sure to select “Use Integration” and select the integration configured in step 1.



Share
Form

Share Form

Server Information

☐ Use Encrypted Connection

Server:

OnBaseTestVM/AppNet

Data Source:

OnBaseTest

Authentication

☐ Use Fixed Credentials ⓘ

User Name:

Password:

Options

Target URL:

☐ Embed Form

1024 x 768

320x240

640x480

800x600

1024x768

☒ Use Integration

Murphy

Result

lwqaZHKnjFMtk0awY%2f00Bijbj5kC8bvjKwgb36D7a%2ferKUwtr5iBU6yRwe34ROSFujLO003Afpzl6GM%3d

Close

INTERNAL ONLY!

Do not distribute outside of DataBank

3. Create parameter string formatted as follows:

```
&ufpre<urlEncodedFieldId_1>=<urlEncodedFieldValue_1>&ufpre<urlEncodedFieldId_1>=<urlEncodedFieldValue_1>& ...  
&ufpre<urlEncodedFieldId_n>=<urlEncodedFieldValue_n>
```

```
1. // Create a string of parameters representing the default-value fields  
2. private static string CreateParameterString(List<KeyValuePair<string, string>> fields)  
3. {  
4.     try  
5.     {  
6.         var builder = new StringBuilder();  
7.         foreach (var field in fields)  
8.         {  
9.             builder.Append($"&ufpre{Uri.EscapeDataString(field.Key)}={Uri.EscapeDataString(field.Value)}");  
10.        }  
11.        return builder.ToString();  
12.    }  
13.    catch (Exception ex)  
14.    {  
15.        throw new ApplicationException("Error creating parameter string!", ex);  
16.    }  
17. }
```

NOTES:

- a. The URL string should contain the '&' in the beginning, as the server expects that format when it reconstructs the string.
- b. When creating the URL to calculate the hash, the constant ufpre should be attached to the beginning of every field ID. This is so that it will be easy to identify the fields that will be pre-populated from the URL string. The field IDs on the form do not need to be changed.
- c. Escape only the ID and value portion of the URL. Do not escape the entire string at once, but just the individual parameters one by one. Escaping the entire URL string would also escape the '&' and '=' characters used to append the field ID and value pairs on to ' the URL. Do not escape the '&' and '=' characters.

INTERNAL ONLY!

Do not distribute outside of DataBank

4. Parse the URL escaped parameter string into UTF-8 format.

```
1. // Get the UTF-8 encoded byte array from the string
2. private static byte[] StringToBytes(string value)
3. {
4.     try
5.     {
6.         return Encoding.UTF8.GetBytes(value);
7.     }
8.     catch (Exception ex)
9.     {
10.        throw new ApplicationException("Error converting string to byte array!", ex);
11.    }
12. }
```

5. Ensure the token is a base64 string. The token should already be a base64 string by default.
6. Calculate the hash of the URL string, which was parsed into UTF-8 format, using the HMAC method and SHA256 algorithm with the token as the secret.
and...
7. Convert the calculated hash into base64 string, escaping the base64 string.

```
1. // Generate the hash string
2. private static string GenerateHash(string token, byte[] parameterBytes)
3. {
4.     try
5.     {
6.         byte[] tokenBytes = Convert.FromBase64String(token);
7.         using (var hmac = new HMACSHA256(tokenBytes))
8.         {
9.             var hashBytes = hmac.ComputeHash(parameterBytes);
10.            return Uri.EscapeDataString(Convert.ToBase64String(hashBytes));
11.        }
12.    }
13.    catch (Exception ex)
14.    {
15.        throw new ApplicationException("Error generating hash from byte array!", ex);
16.    }
17. }
```

INTERNAL ONLY!

Do not distribute outside of DataBank

8. Append the URL Encoded String value into the shared form URL and add the generated hash so that the shared form URL looks similar to the following.

`http://<Host>/AppNet/UnityForm.aspx?dl=<SharedFormHash>&<ParameterString(ufpre...)>&ufprehash=<ParameterHash>`

```
1. // Create the shared form URL
2. private static string GenerateUrl(string baseUrl, string parameterString, string hash)
3. {
4.     try
5.     {
6.         return $"{baseUrl}{parameterString}&ufprehash={hash}";
7.     }
8.     catch (Exception ex)
9.     {
10.        throw new ApplicationException("Error generating shared form URL!", ex);
11.    }
12. }
```